BPFreJIT: A Dynamic and Extensible Compilation Framework for eBPF

Anonymous Author(s)

Abstract

eBPF is widely adopted in modern production systems, yet its in-kernel JIT compilers remain under-optimized compared to mature runtimes like JVM and LLVM. The JIT intentionally trades performance for safety, and closing the gap inside the kernel would require tens of thousands of lines of safety-critical code. We find that even when optimizations are added, statically applying them regresses 68% of real programs because optimization profitability is deployment-dependent. We observe that eBPF's safety is established at the bytecode level: the existing verifier can re-check any rewritten program, and eBPF bytecode is structured enough to serve as an optimization IR. Our key insight is that deployment-time optimization can therefore be safely offloaded to userspace by operating at this level. We present BPFreJIT, a compilation framework that combines runtime specialization with hardware specialization through *kinsn*, an extensible instruction mechanism analogous to LLVM intrinsics with formally verified equivalence between each eBPF expansion and its native lowering. We implement BPFreJIT with fewer than 600 lines of kernel modifications, transparently supporting existing eBPF ecosystem. Evaluation demonstrates that BPFreJIT achieves up to 40% performance improvement, 50% reduction in binary size, and low recompilation overhead. BPFreJIT provides a practical, upstream-compatible path toward extensible compilation frameworks for eBPF.

CCS Concepts: • **Software and its engineering → Just-in-time compilers**.

Keywords: Linux kernel, eBPF, verifier, just-in-time compiler, dynamic optimization

ACM Reference Format:
Anonymous Author(s). 2026. BPFreJIT: A Dynamic and Extensible Compilation Framework for eBPF. In *Proceedings of The 32nd Symposium on Operating Systems Principles (SOSP '26).* ACM, New York, NY, USA, 11 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

eBPF is widely used for performance-critical kernel extensions: networking [1, 2], observability [23], security enforcement [24], and scheduling [4] inside OS kernels [**? ?**]. When loaded into the Linux kernel, each program is verified for safety and JIT-compiled to native code at load time. As eBPF sees increasingly large-scale deployment [**? ?**], native code quality on these hot paths directly affects end-to-end efficiency in cloud platforms.

However, current eBPF compilation is under-optimized. We find that with the same eBPF bytecode, a user-space LLVM JIT backend [**?**] achieves 40% faster execution and 50% smaller native code. For example, the JIT does not exploit hardware-specific instructions or runtime information. Mature runtimes close comparable gaps through extensible compilation frameworks for multiple architectures and profile-guided optimization [13, 18, 21]; the eBPF JIT has none, and existing compile-time optimizers [17, 30, 33] that operate before verification cannot exploit deployment-time opportunities.

This is because the in-kernel JIT [3] trades performance for safety: it performs one-to-one instruction translation on a bytecode interface designed to be straightforward to verify, so the verifier and JIT compiler are simple enough to be fully formally verified [**? ?**].

Closing this gap through existing approaches is impractical. Implementing LLVM-level optimization in the kernel JIT would require tens of thousands of lines of safety-critical code that are difficult to maintain, audit, and upstream. Prior compile-time optimizers [12, 17, 30, 33] improve code before deployment but miss runtime-guided specialization and do not provide transparency. Unlike a standalone application, an eBPF extension is embedded into diverse kernel contexts with varying workloads, hardware, and runtime behavior that a compile-time compiler cannot anticipate. We find that even when legal lowerings are added, statically applying them regresses 68% of real programs.

Our key insight is that deployment-time optimization of kernel eBPF programs can be safely offloaded to userspace by using the *bytecode* as an optimization IR. We observe that eBPF's safety is established at the bytecode level: the existing verifier [11] can re-check any rewritten program with the same guarantees as the original load, and eBPF bytecode is structured enough to serve for rewriting passes. Unlike native-code patching, bytecode rewriting preserves mandatory safety checking; unlike prior pre-load approaches, it operates at deployment time with access to runtime context.

This insight leads to an architecture that decouples optimization *safety* from optimization *correctness*. The kernel provides only a minimal syscall for transparent in-place re-verification and replacement; all optimization policy lives in a userspace daemon that need not be trusted for kernel safety. Invalid rewrites are rejected by the verifier and the original program continues unchanged.

To extend safe optimization to hardware-specific idioms that ordinary eBPF cannot represent, we introduce *kinsn*, an extensible instruction mechanism analogous to LLVM

intrinsics. eBPF bytecode, like any fixed ISA, cannot express platform-specific operations. Kinsn extends the bytecode to close this gap. Each kinsn has two realizations: a canonical expansion to ordinary eBPF that the verifier checks, and an architecture-specific native lowering that the JIT emits. The equivalence between each expansion and its native lowering is formally verified, so the safety properties the verifier establishes on the expansion transfer to the code the JIT actually emits. New kinsn are delivered as trusted kernel modules, requiring no changes to the kernel code.

We propose BPFreJIT, an extensible deployment-time eBPF compilation framework. BPFreJIT enables two complementary classes of optimization: *hardware specialization*, which uses kinsn to emit efficient platform-specific code, and *runtime specialization*, which inlines stable runtime values and adapts when they change. We evaluate BPFreJIT on a corpus of real eBPF programs from production projects. BPFreJIT's deployment-adaptive policy eliminates the regression and achieves up to 40% improvement, with roughly 15 ms recompilation latency per program and ~600 lines of kernel changes.

Contributions. This paper makes three contributions:

- We characterize the code-generation gap between eBPF's kernel JIT and LLVM backends, and show that optimization profitability is deployment-dependent.
- We design and implement BPFreJIT, which offloads optimization to untrusted userspace at the bytecode level, using the kernel verifier as safety gate with minimal kernel changes.
- Evaluation shows that BPFreJIT achieves up to 40% improvement on production workloads while preserving verifier-backed safety.

2 Background

This section introduces the standard eBPF execution pipeline (§2.1) and a concrete example that motivates the need for a richer bytecode ISA (§3.1).

2.1 eBPF JIT

Figure 1 shows the standard eBPF execution path. In the standard deployment path, developers write a BPF program in C, compile it in userspace with Clang/LLVM into an object file containing ordinary eBPF bytecode, and load that bytecode through the `bpf(BPF_PROG_LOAD)` syscall. Before the program executes in kernel context, the verifier perform various safety checks on the submitted bytecode, such as control-flow validity and memory safety [26]. After verification, the kernel performs JIT compilation, which lowers accepted eBPF bytecode to native machine code before execution [6]. The compiled program is then attached to the hook defined by its program metadata. Programs are invoked at runtime by kernel events such as packet arrival. Unlike mature runtimes that employ tiered recompilation and profile-guided optimization [13, 18, 21], the eBPF kernel JIT performs a single, one-shot translation at load time with no subsequent recompilation, runtime feedback, or optimization passes.

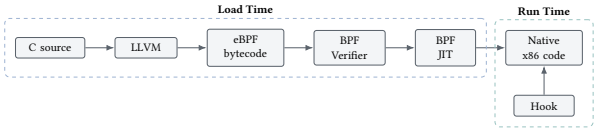

Figure 1. eBPF execution pipeline. C source is compiled to BPF bytecode, verified and JIT-compiled at load time, then executed as native x86 code at runtime via kernel hooks.

3 Motivation

3.1 Motivating Example: 64-bit Rotate

The current compilation pipeline still leaves room for target-specific optimization. A simple example is a 64-bit rotate used in a hash-style computation. At the source level, the programmer expresses one semantic operation: rotate a 64-bit value left by a constant amount, as shown in Figure 2. A userspace toolchain like LLVM optimizes this to use the accelerated `rol` instruction.

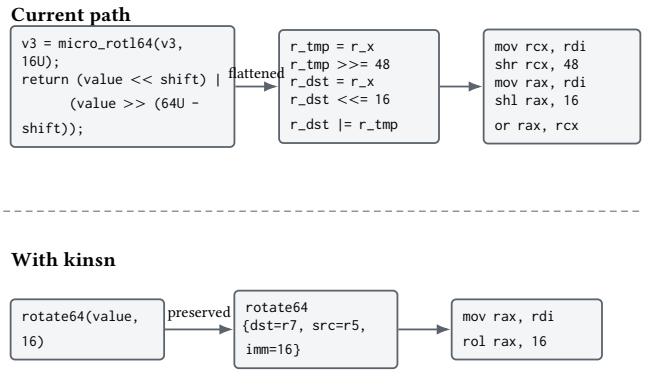

Figure 2. A 64-bit rotate loses semantic structure as it moves through the current pipeline. The source-level rotate idiom is flattened into ordinary eBPF shifts and an `or`; the current x86 JIT therefore emits a longer shift/or sequence instead of the more direct `rol` idiom that the target ISA already supports.

Ordinary eBPF, however, does not contain an optimized bytecode instruction, so cannot preserve that rotate as a first-class instruction. By the time the kernel JIT receives the program, the operation has already been expanded into a sequence of shifts and an `or`. As a result, the native code emitted by the JIT can be longer, less direct and less performant than the native target ISA can support.

The example above illustrates a fundamental limitation of the current pipeline. Ordinary eBPF uses a primitive bytecode ISA, which keeps verifier reasoning simple and makes

programs portable across architectures. But the same bytecode form does not preserve all optimization-relevant source semantics. Once operations (e.g., rotate or select) have been decomposed before entering the kernel, recovering equally efficient native code later would require substantially more analysis and target-specific logic in the in-kernel JIT. The issue, therefore, is that one bytecode form is being used both as a proof form for verification and as the input to native code generation.

The key observation is that using one bytecode form both as the verifier-facing representation and as the input to native code generation creates conflicting design goals. Verification prefers a small, canonical ISA that the kernel can reason about uniformly across targets, whereas code generation benefits from retaining higher-level operations as explicit semantic units until late lowering. The key insight is therefore to decouple them: the bytecode form that preserves optimization intent need not be the same bytecode form that is submitted to the verifier. Section **??** introduces *kinsn*, the bytecode abstraction that realizes this separation.

4 Bytecode-Level Optimization

This section introduces *kinsn*, a bytecode abstraction that separates the representation used to preserve optimization intent from the ordinary-eBPF form used for kernel verification. A kinsn keeps operations such as `rotate64`, `select64`, and `extract64` explicit in the bytecode, instead of forcing the compiler to flatten them early into ordinary-eBPF idioms. For verification, each kinsn is expanded into a semantics-equivalent ordinary-eBPF sequence and checked by the kernel's existing verifier. For execution, the original kinsn is lowered directly to target code while its higher-level semantics remain explicit. This separation keeps sophisticated optimization in userspace, preserves the current kernel verification interface, and avoids pushing target-specific recovery logic into the in-kernel JIT.

We first describe how a kinsn is represented in extended eBPF bytecode (§4.1). We then explain how the same kinsn is interpreted by verification and execution (§4.2). The key point is that these two paths implement the same instruction contract: verification reasons about the instantiated ordinary-eBPF sequence, while JIT code generation may directly emit architecture-specific instructions from the original kinsn representation. §**??** then shows how BPFreJIT constructs and uses kinsns in its rewrite pipeline.

4.1 Representation of Kinsn

Kinsn is a dedicated instruction form in eBPF bytecode. A program may therefore contain explicit kinsns, rather than encoding every optimization opportunity as an ordinary-BPF instruction sequence. This design keeps optimization intent in the bytecode itself, where later compilation stages can still observe and exploit it. Concretely, each kinsn is encoded as a sidecar plus a kinsn call. Ordinary eBPF appears only as an instantiated form: it is used when the kernel needs a verifier-visible proof sequence and when a kinsn must fall back to the generic path.

Kinsn	x86	ARM64
`rotate64`	`ROL/ROR`	`EXTR`
`select64`	`CMOV`	`CSEL`
`extract64`	`BEXTR`	`UBFM`
`endian_load64`	`MOVBE`	`REV`
`speculation_barrier`	`LFENCE`	`DSB SY + ISB`

Table 1. Currently implemented kinsn targets and their x86/ARM64 native lowerings.

Each kinsn is represented by two parts: a *target* and a *sidecar* payload. The target identifies the instruction family, such as `rotate64`, `select64`, `extract64`, or an endian-aware load. The sidecar binds that abstract operation to a concrete program point by encoding the relevant registers, immediate fields, and mode bits. Figure **??** uses `select64` as a running example. In ordinary eBPF, the operation appears as a small branch diamond. In kinsn form, the same operation appears as one instruction kinsn whose target names the operation and whose sidecar carries the destination register, the condition input, and the candidate values. This split is important because it preserves the semantic structure that later lowering wants to realize directly, instead of forcing the JIT to recover that structure from already flattened ordinary-BPF code.

4.2 Kinsn Semantics in Verification and Execution

Each kinsn has one canonical semantics shared by the verifier and the JIT. A module-side descriptor defines how the kinsn is instantiated into an equivalent ordinary-BPF sequence, and it may also provide architecture-specific native emission routines for direct lowering. The instantiated ordinary-BPF sequence is the canonical, verifier-visible meaning of the kinsn. Native lowering does not define a different meaning; it is another realization of the same instruction contract.

The verifier checks kinsn by temporarily lowering each kinsn to its instantiated ordinary-BPF sequence. The kernel replaces the original kinsn with this proof sequence, runs the standard eBPF verifier on the lowered program, and then restores the original kinsn encoding after verification succeeds. In this design, ordinary eBPF serves as the proof form for kinsn. The verifier does not need to reason about target-specific machine code, and kinsn does not require a separate safety checker.

Execution follows a different path while preserving the same semantics. The original kinsn representation is kept until code generation, and the JIT backend dispatches on the target and sidecar payload to emit an architecture-specific native sequence directly from that representation. For example,

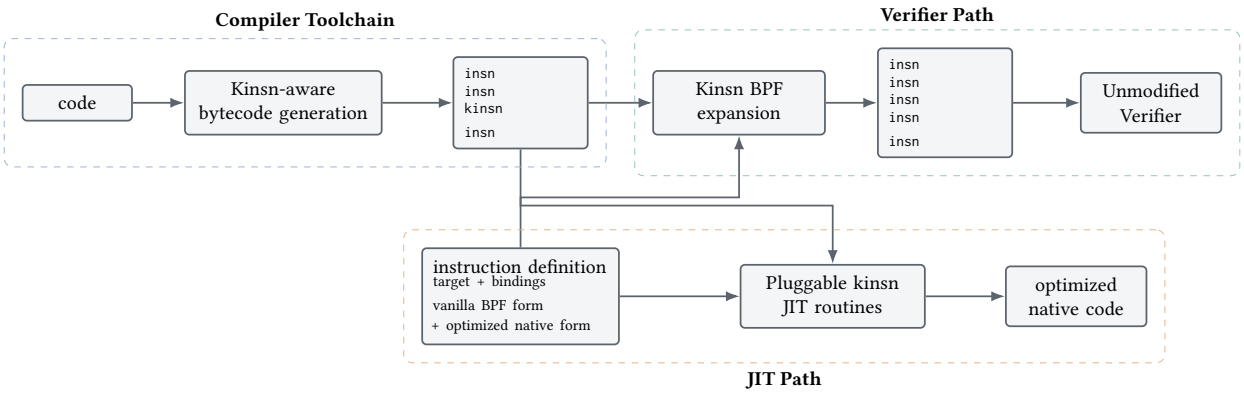

Figure 3. BPF Optimization Pipeline utilizing specialized Kernel Instructions (Kinsns).

the `select64` kinsn in Figure **??** can lower to a conditional-move sequence on x86 rather than re-materializing the branchy ordinary-BPF proof form. When direct native emission is unavailable for a kinsn, the kernel rewrites that kinsn to the same instantiated ordinary-BPF sequence used by the verifier and continues with the generic path. Thus, verification and execution follow different lowering paths, but both remain anchored to the same canonical kinsn semantics.

5 BPFreJIT Design and Implementation

Using kinsn as the optimization-facing program form introduced in Section **??**, BPFreJIT adds a post-load recompilation loop for live eBPF programs. Figure 4 summarizes this loop. After a program is loaded and attached, the kernel keeps the ordinary JIT fast path running and saves the original bytecode as a stable baseline for future rewrites. A privileged userspace daemon discovers a live program, combines that baseline with deployment facts such as runtime profiles and stable values, and rewrites one whole-program candidate. The kernel then re-verifies that candidate, re-JITs it, and replaces the native image in place for the same logical program. This organization gives userspace control over which optimization or hardening candidate to try, while the kernel remains the final authority on legality and safety. The rest of this section states the design contract of this loop, identifies the two implementation constraints that shape it, and then walks through one optimization cycle in detail.

5.1 Design Principles

Preserving the existing safety boundary. BPFreJIT does not bypass the kernel's existing admission path. Userspace may propose a rewritten program, but every candidate is submitted as a complete eBPF program and passes through the normal re-verification and re-JIT path before installation. The kernel therefore continues to enforce the same accept-or-reject safety boundary that already governs `BPF_PROG_LOAD`. If verification or recompilation fails, the candidate is discarded and the current live image remains in place. This

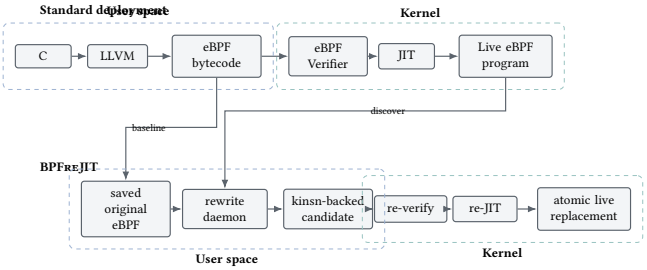

Figure 4. Overview of BPFreJIT. The daemon reads a live program, rewrites a whole program in userspace, and sends it back to the kernel; the kernel re-verifies, re-JITs, and replaces the JIT code in place.

fail-closed organization lets the daemon own rewrite construction without weakening the kernel's safety model.

Preserving deployment transparency. BPFreJIT is designed to optimize already-loaded programs without changing how applications deploy them. Recompilation starts from the saved original bytecode rather than from the currently installed native image, and successful recompilation replaces code in place for the same logical program rather than detaching and reloading a new one. Existing applications, loaders, and attachment paths therefore continue to use the standard eBPF workflow. After installation, execution still follows the ordinary JIT fast path, so the framework adds no steady-state execution overhead beyond the one-time recompilation work.

5.2 Implementation Challenges and Key Insights

There are two implementation challenges define the design of BPFreJIT.

Challenge 1: Recovering a stable rewrite baseline. A live program is not a clean optimization input: verification, fixups, and JIT compilation have already transformed the installed form. BPFreJIT therefore saves the original bytecode at load time and restarts every recompilation round from

that saved copy. This makes each round a whole-program reconstruction from one stable baseline, rather than an incremental patch over an already transformed image.

Challenge 2: Supporting richer code generation without complicating kernel admission. Userspace needs a richer rewrite substrate than ordinary BPF idioms alone, but the kernel still needs one clear contract for checking and realizing rewritten code. BPFreJIT addresses this with kinsn: userspace introduces kinsn-backed realizations where beneficial, while the kernel validates and lowers them through the normal re-admission path. This extends post-load recompilation without turning the kernel JIT into a general optimizer.

5.3 Userspace Rewrite Selection

BPFreJIT assigns rewrite selection to userspace. Whether a rewrite is worthwhile depends on deployment-specific information such as workload behavior, machine characteristics, and runtime values that only become visible after a program has been running. The daemon therefore decides when to revisit a live program and what candidate to try next, while the kernel remains responsible only for deciding whether that candidate is legal and safe.

The daemon always submits one rewritten whole program rather than incremental patches. Whole-program construction lets userspace compose interacting decisions before crossing the kernel boundary. When conditions change, the daemon can revisit the same live program by rebuilding a fresh candidate from the saved original bytecode and resubmitting it through the same kernel admission path. If the new candidate is rejected, the currently running program remains unchanged.

6 Evaluation TO-DO, data not real

In this section, we aim to answer the following research questions:

- **RQ1**: How effectively does BPFreJIT improve the efficiency of generated native code?
- **RQ2**: How much application-level performance benefit does BPFreJIT deliver in realistic deployments?
- **RQ3**: What is the overhead of online re-compilation in BPFreJIT?
- **RQ4**: Does BPFreJIT preserve the existing eBPF safety model and semantic correctness of its rewrites?
- **RQ5**: How easily can BPFreJIT be extended with new optimization capabilities?

We evaluate BPFreJIT along five complementary dimensions. RQ1 examines whether BPFreJIT improves the efficiency of the native code ultimately emitted by the kernel JIT, including both code compactness and low-level execution efficiency. RQ2 asks whether these low-level code improvements translate into measurable application-level benefits in realistic deployments. RQ3 measures the online cost of using BPFreJIT in practice, including the overheads of profiling, re-compilation, and program replacement on live systems. RQ4 evaluates whether BPFreJIT preserves the existing eBPF safety model and whether its rewrites remain semantically consistent with the original programs. RQ5 evaluates whether BPFreJIT is truly extensible by measuring the engineering effort needed to introduce new optimization capabilities.

6.1 Evaluation Setup

We use three baseline points throughout the evaluation: the stock kernel JIT, BPFreJIT, and an LLVM-based upper bound. Our primary machine is a Recent x86-64 workstation-class CPU with 64 GB RAM, and we use an additional ARM64 development board for cross-architecture validation. The software baseline is Linux 7.0-rc2 with the BpfReJIT prototype enabled. The workload suite spans 14 supported-family microbenchmarks, a corpus of 152 live programs collected from 9 eBPF code bases, Tracee and Katran deployments, and safety-oriented negative and differential tests. Unless otherwise noted, we report 10 runs per point; medians with paired before/after comparisons.

6.2 RQ1: Efficiency of Generated Native Code

We first ask whether BPFreJIT improves the native code ultimately emitted by the kernel JIT, and whether the opportunities it targets appear broadly enough to matter beyond hand-written kernels.

Supported-family microbenchmarks. To isolate code-generation effects, we evaluate the subset of microbenchmarks that exercise optimization families currently supported by BPFreJIT and compare the stock kernel JIT with live reJIT through BPFreJIT. Table 2 reports the two native-code outcomes directly: runtime under the stock kernel JIT versus runtime after reJIT, together with the corresponding emitted code size. In the current synthetic placeholder dataset, BPFreJIT achieves a geometric-mean 1.13× speedup and a median 10.5% code-size reduction. The largest wins come from wide-memory simplification and rotate-oriented rewriting, whereas conditional-select remains the most workload-sensitive family. The key point of this experiment is not only that the wins are positive, but that BPFreJIT improves several supported optimization families without changing the ordinary execution path after recompilation.

Family-level contribution. The overall microbenchmark result would be less compelling if it were driven by one narrow pattern, so we regroup the same runs by transform family. Table 3 shows that the gains are distributed across several families. Wide-memory rewrites contribute the largest code shrink, rotate contributes the most stable arithmetic wins, and conditional-select exhibits mixed profitability despite having a positive median result. This breakdown supports a core design argument of the paper: better eBPF code generation is not one missing peephole in the kernel JIT, but a

Table 2. Supported-family microbenchmarks. Runtime is in ns per operation and JIT image size is in emitted native-code bytes; lower is better in both cases. Across these synthetic placeholder results, BPFreJIT achieves a geometric-mean 1.13x speedup over the stock kernel JIT together with a median 10.5% JIT-image reduction, while LLVM remains a stronger upper bound. rq-1-exp-1

	Runtime (ns/op)			JIT image (B)		
Benchmark	Kernel JIT	BpfReJIT	LLVM	Kernel JIT	BpfReJIT	LLVM
load_byte_recompose	95	72.5	66.9	236	184	163
stride_load_16	88	71.0	66.2	208	171	154
ipv4_parse	110	93.2	86.6	248	208	193
rotate64_hash	82	68.3	63.6	192	163	152
pkt_rss_hash	76	67.9	61.3	176	157	144
flow_hash_mix	89	81.7	76.1	188	171	162
switch_dispatch	73	62.9	61.3	152	143	140
binary_search	69	62.7	61.6	144	138	137
bounds_ladder	78	82.1	79.6	168	163	161
extract_u32	84	77.8	74.3	196	176	169
extract_u64	91	82.0	78.4	214	188	180
be32_load	80	74.8	69.6	170	155	148

Table 3. Transform-family summary for the supported microbenchmarks. Each family aggregates three kernels in the current synthetic placeholder dataset. rq-1-exp-2

Family	Median speedup	Median size shrink
Wide memory	1.24x	18.0%
Rotate	1.12x	11.0%
Cond. select	1.10x	4.0%
Extract/endian	1.08x	10.0%

collection of recurring optimization families, some of which are inherently policy-sensitive.

Coverage in real programs. Microbenchmarks establish headroom, but they do not tell us whether the same opportunities appear in realistic programs. We therefore scan the real-program corpus, count eligible rewrite sites by subsystem and transform family, and then measure the code-size deltas of the eligible subset. Figure 5 shows that these opportunities occur across networking, tracing, and security workloads rather than only in synthetic kernels. In the current placeholder dataset, 78 of 152 programs contain at least one eligible site, and the eligible subset still shows a clearly positive median JIT-image reduction. Thus, the code-generation opportunities targeted by BPFreJIT are not artifacts of a benchmark suite; they are also present in realistic deployed programs.

Together, these results support the RQ1 claim: BPFreJIT improves the quality of native code emitted by the kernel JIT, and the opportunities it exploits are common enough to matter in practice.

6.3 RQ2: Application-Level Performance Benefit in Realistic Deployments

RQ1 shows that BPFreJIT improves native code. We next ask whether these lower-level gains survive the transition to full programs and end-to-end deployments.

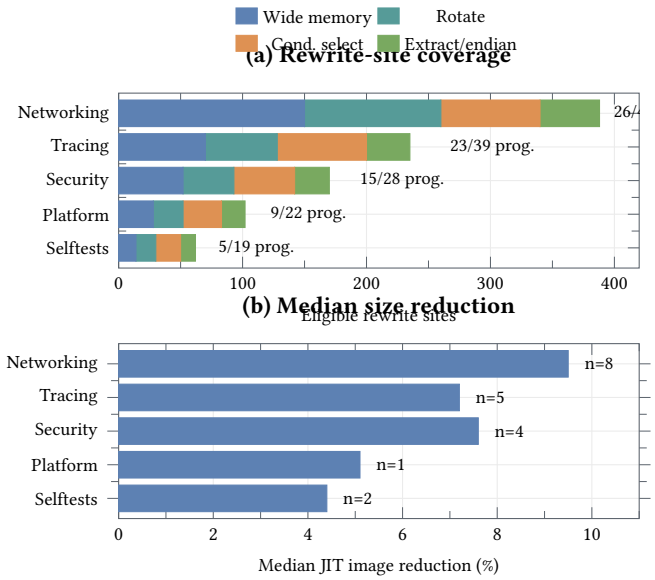

Figure 5. Corpus coverage and code-size benefit. The upper panel groups eligible rewrite sites by subsystem and transform family, with each row annotated by eligible-program coverage; the lower panel shows the median JIT-image reduction among eligible programs in each subsystem. In this synthetic placeholder snapshot, rewrite opportunities appear in 78 of 152 programs, and the eligible subset has a median JIT-image reduction of 6.8%. rq-1-exp-3

Runnable real-program subset. We begin with the runnable subset of the real-program corpus: programs that have reproducible triggers, comparable stock/reJIT measurements, and at least one eligible rewrite site. Figure 6 shows the distribution of per-program speedups on that subset. The median shift remains clearly positive, but smaller than in the microbenchmarks, which is the expected outcome once helper cost, map access, and surrounding program logic are

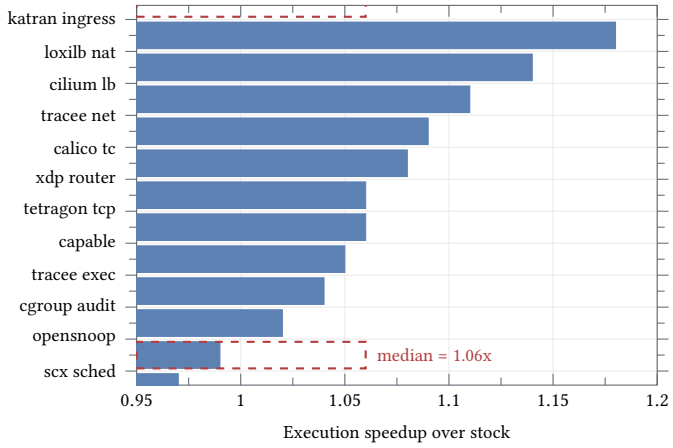

Figure 6. Per-program speedups on the runnable real-program subset. The synthetic placeholder distribution has a median 1.06x speedup and no catastrophic outliers. rq-2-exp-1

included. In the current placeholder dataset, the median speedup is 1.06×. This experiment therefore plays an important bridging role: it shows that the benefit of BPFreJIT survives the move from isolated kernels to full real programs, even though the effect size naturally compresses.

End-to-end deployments. We then evaluate two representative deployment settings: Tracee for observability and security, and Katran for packet processing on the fast path. Table 4 summarizes the compact throughput/BPF/CPU view of the current placeholder results. In the Tracee workloads, lower BPF execution time translates into moderate throughput gains while leaving event rates effectively unchanged; in the current placeholder dataset, event-rate deltas stay within ±0.2%. In the Katran workloads, reduced BPF time per packet yields smaller but still repeatable improvements in throughput and CPU utilization, while tail latency also improves modestly; the current placeholder P99 reduction ranges from 1.2% to 2.1%. The important lesson is not that every deployment sees the same magnitude of gain, but that lower-level code-generation improvements continue to produce measurable benefit at the application level.

Overall, the RQ2 experiments support the claim that BPFreJIT's code-generation improvements are visible above the native-code level and can produce measurable benefit in realistic deployments.

6.4 RQ3: Overhead of Online Re-Compilation

The previous subsections show that reJIT can help. We now ask whether its cost is low enough for online use.

Request latency and pipeline breakdown. Our first overhead experiment compares two ways of invoking reJIT: launching a one-shot daemon process per request versus sending requests to a long-running daemon server. In the current

Table 4. End-to-end deployment summary for Tracee and Katran. Positive throughput values are better; negative BPF time and CPU deltas are better. rq-2-exp-2

Deploy.	Workload	Thr.	BPF	CPU
Tracee	proc_storm	+5.8%	-9.6%	-4.2%
Tracee	file_churn	+3.7%	-7.4%	-3.1%
Tracee	net_churn	+4.9%	-8.8%	-3.7%
Katran	10G_mix	+1.4%	-8.8%	-2.3%
Katran	25G_mix	+1.9%	-9.5%	-2.8%

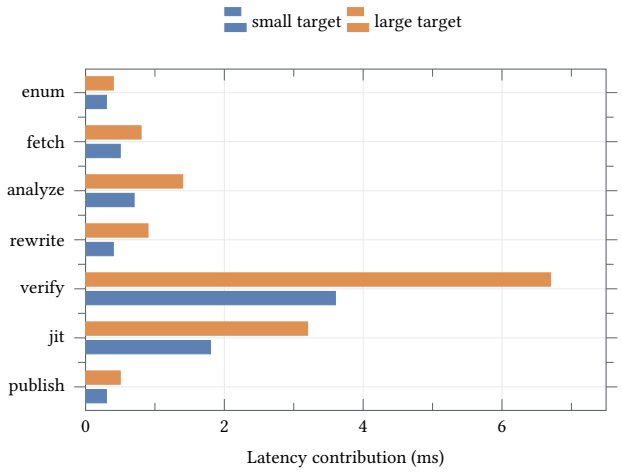

Figure 7. Per-operation reJIT latency breakdown for small and large targets. In both cases, verifier and JIT dominate the end-to-end cost after request dispatch is amortized. rq-3-exp-1

placeholder dataset, one-shot requests take 23.4 ms for a small target and 31.8 ms for a large target, while the persistent server reduces those medians to 7.2 ms and 10.9 ms. Figure 7 then breaks the amortized reJIT latency into its main stages for small and large targets separately. In both cases, verification and JIT account for most of the end-to-end cost, whereas userspace analysis and rewriting remain a smaller fraction. This is the overhead profile we want: once the daemon is resident, the dominant cost of reJIT is still the kernel's existing enforcement path, not a heavyweight userspace optimizer loop.

Steady-state daemon cost. Low request latency alone is not enough; the daemon must also be cheap to leave enabled during normal operation. We therefore compare four daemon states on stable Tracee and Katran workloads: disabled, watch only, profile only, and the full adaptive path. Table 5 shows that watch-only and profile-only modes are close to free, while even the full adaptive mode remains lightweight relative to the performance it can recover. In the placeholder dataset, the adaptive mode stays near 1% daemon CPU and

31 MB RSS. This result supports the decision to keep policy in userspace: the framework imposes little steady-state tax unless we intentionally enable more expensive adaptive behavior.

Transient disruption during live replacement. Finally, we look at whether a single live reJIT event visibly disturbs an ongoing workload. This experiment matters because it tests the operational consequence of in-place replacement: even when recompilation is successful, it must not tear down the running deployment or induce a prolonged disruption.

In the current placeholder trace, a single live replacement causes at most a 4.4% throughput dip and a 5.1% tail-latency spike. Both metrics return to within 0.5% of baseline by 300 ms, and fully return to baseline by 400 ms. rq-3-exp-3

Taken together, the RQ3 experiments show that the online cost of BPFreJIT is low enough to make post-load optimization practical in a continuous deployment setting.

6.5 RQ4: Safety Preservation and Semantic Correctness

The paper's central design claim is that safety and optimization policy are separated: the kernel verifier remains the safety boundary, while userspace is responsible for rewrite correctness. RQ4 evaluates whether that separation holds up in practice.

Fail-safe rejection. We first test the fail-safe property directly. The targeted adversarial suite submits rewrites that attempt to violate memory safety, control-flow integrity, type constraints, privilege rules, and metadata invariants; the fuzzing experiment then broadens that stress with large numbers of randomized bytecode mutations.

In the current placeholder dataset, all 30 targeted invalid rewrites across 5 adversarial categories are rejected, and the original live program remains unchanged in every case. In the randomized stress test, 9983 of 10000 mutated candidates are rejected; the remaining 17 remain verifier-acceptable, while the original live image still stays intact throughout the run.

These are exactly the failure semantics that BPFreJIT is designed to provide: a buggy or compromised optimizer can fail closed, but it does not bypass the verifier.

Semantic consistency. Fail-safe rejection alone does not show that successful rewrites preserve behavior. We therefore compare original and rewritten programs on identical inputs and initial state, checking externally visible results such as return values, map updates, and emitted events. Table 6 summarizes the coverage of this differential suite across rewrite families and representative real programs. In the current placeholder dataset, we observe no mismatches across 53 programs and 94,000 test inputs. The intended interpretation is careful but important: the verifier guarantees safety, while semantic preservation is supported empirically through testing.

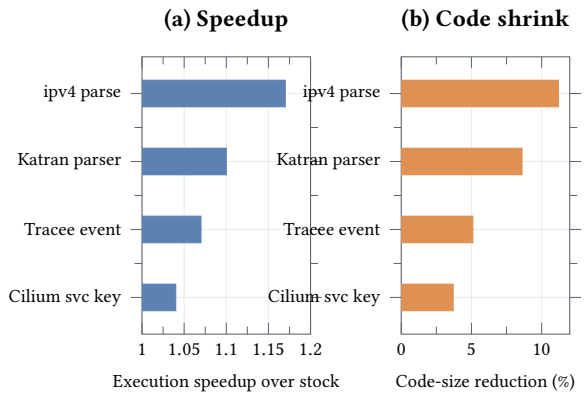

Figure 8. Benefit of one newly added capability. In this synthetic placeholder example, the new endian-fusion family yields a clear win on its dedicated microbenchmark and smaller but still visible gains on real programs. rq-5-exp-2

Overall, RQ4 supports the paper's core separation of responsibilities: the kernel continues to decide safety, and userspace rewrite correctness is validated through systematic testing plus fail-safe rejection.

6.6 RQ5: Extensibility of the Optimization Framework

The final question is whether BPFreJIT is truly a framework or merely a fixed bundle of optimizations.

Adding one new capability. To test extensibility, we add one new optimization family and measure both the implementation effort and the resulting benefit. Table 7 reports only the implementation footprint. In the current placeholder dataset, the work is concentrated in the daemon pass, the optional x86/ARM64 lowering modules, and a small amount of validation and benchmark plumbing, while the core kernel verifier and syscall path remain unchanged. This is the extensibility pattern the design is aiming for: most innovation should happen outside the trusted kernel core.

Value of the new capability. Extensibility would be less meaningful if newly added capabilities were easy to implement but hard to realize in practice. Figure 8 therefore evaluates the same new family on one dedicated microbenchmark and several real programs that expose the corresponding opportunity. The placeholder results show a clear win on the dedicated kernel together with smaller but still positive gains on real programs. This is the right framework-level story: new capabilities can be added with localized effort and then exercised through the same live reJIT workflow as the original ones.

Graceful degradation. Finally, we evaluate the lifecycle behavior of optional capability modules across the key deployment transitions that matter in practice.

We exercise three lifecycle states for an optional capability module. When the module is loaded, the rewrite applies

Table 5. Steady-state daemon cost across workloads and operating modes. rq-3-exp-2

Workload	Mode	Daemon CPU	RSS (MB)	Throughput Δ	Latency Δ
Tracee	disabled	0.0%	0	+0.0%	+0.0%
Tracee	watch	0.4%	18	+0.1%	+0.0%
Tracee	profile	0.7%	23	-0.3%	+0.4%
Tracee	adaptive	1.3%	31	+4.2%	-2.1%
Katran	disabled	0.0%	0	+0.0%	+0.0%
Katran	watch	0.3%	18	+0.0%	+0.1%
Katran	profile	0.6%	24	-0.2%	+0.3%
Katran	adaptive	1.1%	31	+1.6%	-1.4%

Table 6. Differential semantic testing coverage. rq-4-exp-2

Family	Programs	Test inputs	Mismatches
Wide memory	14	28000	0
Rotate	9	18000	0
Cond. select	10	20000	0
Extract/endian	8	16000	0
Real programs	12	12000	0
Total	53	94000	0

Table 7. Engineering cost of one new optimization family. rq-5-exp-1

Component	LOC	Files
Daemon pass	218	4
x86 module	94	2
ARM64 module	86	2
Core kernel	0	0
Tests/bench	73	3
Total	471	11

normally. When it is absent, the daemon skips the optimization and leaves the stock image unchanged. If the module is removed after deployment, the next optimization attempt is rejected while the current live image remains runnable. These cases show that optional capabilities fail gracefully rather than destabilizing the live deployment.

Thus, the RQ5 experiments support the final claim of the paper: BPFreJIT remains extensible in the intended sense, allowing new optimization families to be added and deployed without turning the verifier or JIT into a monolithic in-kernel optimizer.

7 Related Work

Verification and extensibility. Prior work on eBPF verification and extensibility primarily changes how extensions are admitted, validated, hardened, or exposed, rather than how already-accepted live programs are optimized. One line of work strengthens reasoning in the load path through more precise analysis, proof-carrying acceptance, or two-stage verification [8, 19, 25, 27, 29]. A second line increases trust in the execution pipeline by validating, fuzzing, or hardening the verifier, JIT, and runtime [7, 10, 14–16, 20, 22, 28]. Closest to our implementation concerns, ePass makes the BPF load path extensible through in-kernel passes, while Rex and other recent extension systems broaden the safe extension interface itself [5, 9, 31, 32]. These systems make extension admission or execution safer and more expressive; BPFreJIT instead keeps the stock Linux verifier/JIT path intact and revisits already-loaded live programs for transparent post-load re-optimization.

BPF code optimization. Prior direct BPF optimization work instead improves object-level bytecode before deployment. Search-based systems such as K2 and EPSO optimize BPF by discovering better instruction sequences or reusable rewrite rules [30, 33]. Merlin attacks the same problem through a richer compiler pipeline, lifting BPF into LLVM IR to enable broader static optimization [17]. Despite these technical differences, all these systems share the same optimization point: they assume build- or load-path control and access to the original `.bpf.o`, so they cannot transparently revisit programs that were already loaded by third-party tools or exploit runtime behavior that emerges only after deployment [17, 30, 33]. BPFreJIT is complementary because it optimizes at a different point in the lifecycle: post-load, on live programs, with runtime guidance and backend-aware kernel instruction support.

incorporate original LLVM / JVM / Wasm section in background

8 Conclusion

In this paper, we identified a gap in the current eBPF stack: existing optimization techniques improve programs before or during loading, but they do not transparently revisit already-loaded live programs using post-deployment behavior. We presented BPFreJIT, a dynamic and extensible framework that closes this gap through minimal in-kernel support for recovering, re-verifying, and replacing loaded programs, together with a userspace daemon for runtime-guided rewriting and *kinsn* for backend-aware instruction support. Our evaluation across policy-sensitive microbenchmarks, real-program corpora, and end-to-end workloads shows that

BPFreJIT preserves the stock BPF safety model and existing deployment paths while delivering better performance, smaller code, and low recompilation overhead. These results show that post-load optimization of live eBPF programs is practical and effective within the current Linux deployment model.

References

[1] Maxim Bertin, Zakir Durumeric, and Vern Paxson. 2019. Katran: A high performance layer 4 load balancer. In *USENIX NSDI*.

[2] Daniel Borkmann, Starovoitov Alexei, et al. 2016. Cilium: BPF and XDP for containers. In *Linux Plumbers Conference*.

[3] Linux Kernel Community. 2024. Linux Kernel BPF JIT Compiler (arch/x86/net/bpf_jit_comp.c). https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git.

[4] Linux Kernel Community. 2024. sched_ext: BPF extensible scheduler class. https://lwn.net/Articles/922405/.

[5] Kumar Kartikeya Dwivedi, Rishabh Iyer, and Sanidhya Kashyap. 2024. Fast, Flexible, and Practical Kernel Extensions. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) *(SOSP '24)*. Association for Computing Machinery, New York, NY, USA, 249–264. https://doi.org/10.1145/3694715.3695950

[6] eBPF.io Authors. [n. d.]. What is eBPF? — JIT Compilation. https://ebpf.io/what-is-ebpf/#jit-compilation. Accessed: 2025-03-25.

[7] Luis Gerhorst, Henriette Herzog, Peter Wägemann, Maximilian Ott, Rüdiger Kapitza, and Timo Hönig. 2024. VeriFence: Lightweight and Precise Spectre Defenses for Untrusted Linux Kernel Extensions. In *Proceedings of the 27th International Symposium on Research in Attacks, Intrusions and Defenses* (Padua, Italy) *(RAID '24)*. Association for Computing Machinery, New York, NY, USA, 644–659. https://doi.org/10.1145/3678890.3678907

[8] Elazar Gershuni, Nadav Amit, Arie Gurfinkel, Nina Narodytska, Jorge A. Navas, Noam Rinetzky, Leonid Ryzhyk, and Mooly Sagiv. 2019. Simple and precise static analysis of untrusted Linux kernel extensions. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Phoenix, AZ, USA) *(PLDI 2019)*. Association for Computing Machinery, New York, NY, USA, 1069–1084. https://doi.org/10.1145/3314221.3314590

[9] Jinghao Jia, Ruowen Qin, Milo Craun, Egor Lukiyanov, Ayush Bansal, Minh Phan, Michael V. Le, Hubertus Franke, Hani Jamjoom, Tianyin Xu, and Dan Williams. 2025. Rex: Closing the language-verifier gap with safe and usable kernel extensions. In *Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC '25)*. USENIX Association. https://www.usenix.org/conference/atc25/presentation/jia

[10] Di Jin, Alexander J. Gaidis, and Vasileios P. Kemerlis. 2024. BeeBox: Hardening BPF against Transient Execution Attacks. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 613–630. https://www.usenix.org/conference/usenixsecurity24/presentation/jin-di

[11] The kernel development community. 2026. eBPF verifier. https://docs.kernel.org/bpf/verifier.html.

[12] Hsuan-Chi Kuo, Kai-Hsun Chen, Yicheng Lu, Dan Williams, Sibin Mohan, and Tianyin Xu. 2022. Verified programs can party: optimizing kernel extensions via post-verification merging. In *Proceedings of the Seventeenth European Conference on Computer Systems* (Rennes, France) *(EuroSys '22)*. Association for Computing Machinery, New York, NY, USA, 283–299. https://doi.org/10.1145/3492321.3519562

[13] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. *International Symposium on Code Generation and Optimization (CGO)* (2004), 75–86.

[14] Soo Yee Lim, Xueyuan Han, and Thomas Pasquier. 2023. Unleashing Unprivileged eBPF Potential with Dynamic Sandboxing. arXiv:2308.01983 [cs.OS] https://arxiv.org/abs/2308.01983

[15] Soo Yee Lim, Tanya Prasad, Xueyuan Han, and Thomas Pasquier. 2024. SafeBPF: Hardware-assisted Defense-in-depth for eBPF Kernel Extensions. In *Proceedings of the 2024 on Cloud Computing Security Workshop* (Salt Lake City, UT, USA) *(CCSW '24)*. Association for Computing Machinery, New York, NY, USA, 80–94. https://doi.org/10.1145/3689938.3694781

[16] Tao Lyu, Kumar Kartikeya Dwivedi, Thomas Bourgeat, Mathias Payer, Meng Xu, and Sanidhya Kashyap. 2025. eBPF Misbehavior Detection: Fuzzing with a Specification-Based Oracle. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) *(SOSP '25)*. Association for Computing Machinery, New York, NY, USA, 701–718. https://doi.org/10.1145/3731569.3764797

[17] Jinsong Mao, Hailun Ding, Juan Zhai, and Shiqing Ma. 2024. Merlin: Multi-tier Optimization of eBPF Code for Performance and Compactness. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (La Jolla, CA, USA) *(ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 639–653. https://doi.org/10.1145/3620666.3651387

[18] Benedikt Meurer and Ben L. Titzer. 2015. TurboFan: A new code generation architecture for V8. Google Munich Technical Talk. https://docs.google.com/presentation/d/1sOEF4MlF7LeO7uq-uThJSuIJlTh--wgLeaVibsbb3tc.

[19] Luke Nelson et al. 2025. BCF: Certificate-Backed BPF Verification. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP)*. ACM.

[20] Luke Nelson, Jacob Van Geffen, Emina Torlak, and Xi Wang. 2020. Specification and verification in the field: applying formal methods to BPF just-in-time compilers in the Linux kernel. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation (OSDI'20)*. USENIX Association, USA, Article 3, 21 pages.

[21] Michael Paleczny, Christopher Vick, and Cliff Click. 2001. The Java HotSpot™ Server Compiler. In *Proceedings of the Java Virtual Machine Research and Technology Symposium (JVM)*. 1–12.

[22] Chaoyuan Peng, Muhui Jiang, Lei Wu, and Yajin Zhou. 2024. Toss a Fault to BpfChecker: Revealing Implementation Flaws for eBPF runtimes with Differential Fuzzing. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security* (Salt Lake City, UT, USA) *(CCS '24)*. Association for Computing Machinery, New York, NY, USA, 3928–3942. https://doi.org/10.1145/3658644.3690237

[23] Aqua Security. 2024. Tracee: Runtime security and forensics using eBPF. https://github.com/aquasecurity/tracee.

[24] KP Singh. 2020. BPF LSM: MAC and Audit policy using eBPF. https://lwn.net/Articles/808048/.

[25] Hao Sun and Zhendong Su. 2024. Validating the eBPF Verifier via State Embedding. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 615–628. https://www.usenix.org/conference/osdi24/presentation/sun-hao

[26] The Linux Kernel Community. [n. d.]. eBPF Verifier. The Linux Kernel Documentation. https://docs.kernel.org/bpf/verifier.html Accessed: 2025-03-25.

[27] Harishankar Vishwanathan, Matan Shachnai, Srinivas Narayana, and Santosh Nagarakatte. 2023. Verifying the Verifier: eBPF Range Analysis Verification. In *Computer Aided Verification: 35th International Conference, CAV 2023, Paris, France, July 17–22, 2023, Proceedings, Part III* (Paris, France). Springer-Verlag, Berlin, Heidelberg, 226–251. https://doi.org/10.1007/978-3-031-37709-9_12

[28] Xi Wang, David Lazar, Nickolai Zeldovich, Adam Chlipala, and Zachary Tatlock. 2014. Jitk: a trustworthy in-kernel interpreter infrastructure. In *Proceedings of the 11th USENIX Conference on Operating Systems*

Design and Implementation (Broomfield, CO) *(OSDI'14)*. USENIX Association, USA, 33–47.

[29] Xiwei Wu, Yueyang Feng, Tianyi Huang, Xiaoyang Lu, Shengkai Lin, Lihan Xie, Shizhen Zhao, and Qinxiang Cao. 2025. VEP: a two-stage verification tool chain for full eBPF programmability. In *Proceedings of the 22nd USENIX Symposium on Networked Systems Design and Implementation* (Philadelphia, PA, USA) *(NSDI '25)*. USENIX Association, USA, Article 16, 23 pages.

[30] Qiongwen Xu, Michael D. Wong, Tanvi Wagle, Srinivas Narayana, and Anirudh Sivaraman. 2021. Synthesizing safe and efficient kernel extensions for packet processing. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference* (Virtual Event, USA) *(SIGCOMM '21)*. Association for Computing Machinery, New York, NY, USA, 50–64. https://doi.org/10.1145/3452296.3472929

[31] Yuhong Zheng et al. 2025. ePass: Extensible BPF Optimization Passes in the Kernel. In *Linux Plumbers Conference*.

[32] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. 2025. Extending applications safely and efficiently *(OSDI '25)*. USENIX Association, USA, Article 31, 18 pages.

[33] Qian Zhu, Yuxuan Liu, Ziyuan Zhu, Shangqing Liu, and Lei Bu. 2025. EPSO: A Caching-Based Efficient Superoptimizer for BPF Bytecode. arXiv:2511.15589 [cs.SE] https://arxiv.org/abs/2511.15589